

```

import pandas as pd

from w6_indici_diversitate import diversitate

# pandas = o biblioteca definita peste numpy, pentru operatii peste date
# tabelare (randuri si coloane)
# pune la dispozitie 2 tipuri:
# 1. Series - date unidimensionale (coloane in foaia de calcul)
# 2. DataFrame - date bidimensionale (foaia de calcul)

s = pd.Series([10, 20, 30], index=['a', 'b', 'c'])

df = pd.DataFrame({'Name': ['Alice', 'Bob', 'Carol'], 'Age': [28, 32, 25],
'Salary': [5000, 6000, 7000]})

df = pd.read_csv('res/employees.csv', index_col=1)

# proprietati pe data frame
print("DF head\n", df.head()) # primele 5 randuri din df
print("DF tail\n", df.tail()) # ultimele 5 randuri din df
print("DF info\n", df.info()) # structura df: tipuri de date, valori null
print("DF describe\n", df.describe()) # statistici descriptive pe coloane
print("DF shape", df.shape) # (rows, columns)
print("DF columns", df.columns) # un obiect ce contine lista tuturor
coloanelor
print("DF index", df.index) # un obiect ce contine lista tuturor
referintelor randurilor

# accesul datelor - se foloseste operatorul de indexare [rows , columns]
# citirea datelor pe coloane
ages = df["Age"]

# coloane = ["Name", "Salary"]
# subset = df[df.columns - ["Name", "Salary"]]
subset = df[["Gender", "Salary"]]

# citirea datelor pe randuri folosind indecsi | iloc - index location
fr = df.iloc[0] # primul rand din df
ftr = df.iloc[0:3] # primele 3 randuri

# citirea datelor pe randuri folosind etichete/labels | loc
# probabil mult mai intuitiv daca index_col = 1
# label1 = df.loc['Bob'] # cauta randul pentru care referinta (index value)
# = Bob
# label2 = df.loc['Eva':'Ivy', ["ID", "Salary"]]
# print(label1)
# print(label2)

label1 = df.loc["Bob"] # cauta randul pentru care referinta (index value)
# = 1
label2 = df.loc["Eva":"Ivy", ["Gender", "Salary"]]

# modificarea datelor
df["TaxedSalary"] = df["Salary"] * 0.9 # adaugarea unei coloane noi

# redenumire de coloane
df.rename(columns={'Salary': 'GrossSalary'}, inplace=True)
df.rename(columns={'GrossSalary': 'Salary'}, inplace=True)

# stergere de randuri sau coloane
df.drop(columns=["TaxedSalary"], inplace=True)

```

```

df.drop(index=["Carol"], inplace=True)

# data sanitization = curatarea datelor - gestionarea celurilor goale din
data frame
# in general veti avea de a face cu unul din urmatoarele scenarii:
# 1 - operam cu date numerice
# 2 - operam cu date categorice (strings)
# in sanitizarea datelor fie alegem sa facem drop pe randuri sau coloane,
fie substituim celulele
# lipsa cu valori convenabil alese
# - date numerice: de regula folosim media pe coloana sau o valoare
potrivita in context
# - date categorice: de regula folosim modulul pe coloana (cea mai frecvent
intalnita valoare)
#                               sau o valoare potrivita in context

missing = df.isna().sum()

# replace
value = df["Salary"].mean()
df.fillna({"Salary": value}, inplace=True)

df.fillna(0)

# drop
df.dropna() # drop fiecarui rand care are NaN. echivalent:
df.dropna(axis=0)
df.dropna(axis=1) # drop fiecarei coloane care are NaN

# transformari
# vectorizate
df["AgeInMonths"] = df["Age"] * 12

# lambda si apply
# def return_bracket(x):
#     if x > 6000:
#         return "High"
#     return "Low"
# df["IncomeBracket"] = df["Salary"].apply(return_bracket)

df["IncomeBracket"] = df["Salary"].apply(lambda x: "High" if x > 6000 else
"Low")

# metode din clasa string
df["Gender"] = df["Gender"].str.lower()

# statistici
# centrare - repositionarea datelor in jurul unei valori centrale, care de
regula este 0. in urma procesului de
# centrare a datelor, media devine 0
df["Salary_centered"] = df["Salary"] - df["Salary"].mean()

# scalare - aducerea intervalelor de valori ale datelor la un ordin de
marime comparabil

# standardizarea datelor
# standardizarea = centrare + scalare
# procedeul de standardizare se aplica atunci cand operati cu algoritmi ce
au ca ipoteza faptul ca datele respecta o
# distributie normala: ACP, regresile
# in urma standardizarii, media  $\bar{x} = 0$ , iar  $S_x = 1$ 

```

```

# standardizarea functioneaza ca o translatare si redimensionare a datelor
df["Salary_standardized"] = (df["Salary"] - df["Salary"].mean()) /
df["Salary"].std()

# normalizarea datelor
# este un procedeu ce transforma datele folosind o formula precum: (Xi -
Xmin) / (Xmax - Xmin)
# in urma normalizarii datelor, valorile obtinute vor fi in intervalul
[0:1]
# normalizarea datelor, transforma reprezentarea acestora - in urma
normalizarii, reprzentara grafica difera
# normalizarea se utilizeaza in general in ML si in algoritmi unde datele
au domenii de definitie finite
# (procesare de imagini)

# statistici descriptive
df["Salary"].mean()
df["Salary"].median() # valoarea care imparte setul de date in 2 jumatati
egale
df["Salary"].mode() # in romana: modul. valoarea cea mai frevent intalnita
pe colona

# metrice privitoare la dispersia datelor
df["Salary"].std()
df["Salary"].var()

# relatia intre 2 variabile
# valoarea coef este pozitiva: intre cele 2 variabile exista o relatie
directa - atunci cand 1 var creste si cealalta
# creste, respectiv invers
# valoarea coef este negativa: intre cele 2 variabile exista o relatie de
inversa proportionalitate - atunci cand 1
# var creste, cealalta scade, respectiv invers
# daca valoarea tinde sa fie apropiata de 0, cele 2 variabile tind sa fie
independente
df[["Age", "Salary"]].corr()

# desenare grafice direct din pandas
# df["Age"].hist(bins=15, edgecolor='red')
df["Age"].plot()
# plt.show()

# merge pe tabele (echivalentul unui join in SQL) si concatenare
df1 = pd.DataFrame({"ID": [1, 2, 3], "Name": ["Alice", "Bob", "Carol"]})

df2 = pd.DataFrame({"ID": [4, 5, 6], "Name": ["Mark", "Eva", "Ivy"]})

df3 = pd.DataFrame({"ID": [1, 2, 3], "Department": ["IT", "HR",
"Finance"]})

merged = df1.merge(df3, on="ID")
print(merged)

concat = pd.concat([df1, df2])
print(concat)

# tipuri de merge - vom presupune ca operam cu 2 data frames: df1 si df2,
iar operatiile de merge respecta formatul
# df1.merge(df2)
# inner - intersectia dintre df1 si df2, adica randurile comune celor 2
surse

```

```

# left - toate randurile din df1, chiar daca nu au un corespondent in df2
# right - toate randurile din df2, chiar daca nu au un corespondent in df1
# outer - reuniunea dintre df1 si df2 sau toate datele

employees = pd.read_csv('res/employees.csv')
departments = pd.read_csv('res/departments.csv')

# exemplele de mai jos sunt cazuri de merge pe baza coloanelor
# merge pe baza de coloane presupune ca in ambele data frames exista o
coloana comuna (in df.columns, nu df.index!!!)

# inner
inner = employees.merge(departments, on="DepartmentID", how="inner")
print(inner)

# left
left = employees.merge(departments, on="DepartmentID", how="left")
print(left)

# right
right = employees.merge(departments, on="DepartmentID", how="right")
print(right)

# outer
outer = employees.merge(departments, on="DepartmentID", how="outer")
print(outer)

# merge pe baza de index
tabel_etnii = pd.read_csv('res/Ethnicity.csv', index_col=0)
# nan_replace()

variabile_etnii = list(tabel_etnii.columns)[1:]

# calcul populatie pe etnii la nivel de judet
localitati = pd.read_excel('res/CoduriRomania.xlsx', index_col=0,
sheet_name='Localitati')

t1 = tabel_etnii.merge(right=localitati, right_index=True, left_index=True)
print(t1)

g1 = t1[variabile_etnii + ["County"]].groupby("County").agg("sum")
print(g1)

# calcul populatie pe etnii la nivel de regiune
judete = pd.read_excel('res/CoduriRomania.xlsx', index_col=0,
sheet_name='Judete')

t2 = g1.merge(right=judete, right_index=True, left_index=True)
print(t2)

g2 = t2[variabile_etnii + ["Regiune"]].groupby("Regiune").agg("sum")
print(g2)

# calcul populatie pe etnii la nivel de macroregiune
regiuni = pd.read_excel('res/CoduriRomania.xlsx', index_col=0,
sheet_name='Regiuni')

t3 = g2.merge(right=regiuni, right_index=True, left_index=True)
print(t3)

g3 = t3[variabile_etnii +

```

```
[ "MacroRegiune" ]].groupby("MacroRegiune").agg("sum")
print(g3)

g1.to_csv('res/output_etnii_judete.csv')
g2.to_csv('res/output_etnii_regiuni.csv')
g3.to_csv('res/output_etnii_macroregiuni.csv')

# indici de diversitate la nivel de localitate
diversitate_loc = tabel_etnii.apply(func=diversitate, axis=1,
denumire_coloana="Localitate")
diversitate_loc.to_csv('res/diversitate_etnii_localitati.csv')

# indici de diversitate la nivel de judet
diversitate_judet = g1.apply(func=diversitate, axis=1)
diversitate_judet.to_csv('res/diversitate_etnii_judet.csv')

# indici de diversitate la nivel de regiune
diversitate_regiune = g2.apply(func=diversitate, axis=1)
diversitate_regiune.to_csv('res/diversitate_etnii_regiune.csv')
```